

Conexión a MongoDB usando un ODM

Un ODM (Object Document Modeling) es un enfoque o patrón que agrega una capa de abstracción sobre el driver nativo. En lugar de interactuar directamente con el motor de base de datos, las operaciones se realizan mediante modelos, esquemas y métodos propios de la librería del ODM. Esto proporciona un enfoque más cercano al desarrollo orientado a objetos, con validaciones y estructuras definidas en el código.

Ventajas de usar un ODM

- Modelado con esquemas definidos para cada colección: Se definen tipos de datos, validaciones, valores por defecto, índices, middlewares, métodos personalizados. El modelo funciona como un objeto que representa a la colección, haciendo que esté integrado directamente en el código.
- Simplificación en la construcción de consultas: Se suele tener a disposición distintos métodos que simplifican las consultas en comparación al driver nativo. Por ejemplo, se tienen métodos como *model.findByIdAndUpdate(id, actualización)*.
- Facilidad en el mantenimiento de aplicaciones grandes: Al establecerse una forma de trabajo más estructurada y unificada, se evitan estructuras inconsistentes y validaciones repetidas que pueden surgir del trabajo en equipo, es decir, fuerza orden y coherencia en toda la aplicación.
- Abstracciones útiles: Permite de manera sencilla el uso de middlewares, relacionado de colecciones y manejo de tipos más complejos, que de otra manera deben ser implementadas manualmente en el código fuente.

Situaciones ideales para el driver nativo

El uso de un ODM es recomendable en escenarios como:

- Aplicaciones con reglas de negocio complejas, donde se necesita forzar una estructura de datos coherente
- APIs donde cada recurso de la BD se define naturalmente como un modelo
- Proyectos de mediana o gran escala donde varios equipos trabajan sobre la misma base de datos
- Aplicaciones donde la validación, índices y lógica de datos deben estar declaradas explícitamente en el código, o en ambos contextos

Si bien existe un costo de rendimiento adicional respecto al driver nativo, el beneficio de organización y abstracción suele compensarlo en la mayoría de aplicaciones.

Ejemplo de uso para NodeJS

El ODM más utilizado en el ecosistema de NodeJS es [mongoose](#). Para usarlo en un proyecto de NodeJS, primero se debe inicializar el proyecto y a continuación instalar el paquete mediante el gestor npm. Para hacerlo corremos lo siguiente en la terminal:

```
npm init --init-type module -y
npm install mongoose
```

Luego, en el archivo de inicio de la aplicación se puede hacer uso del driver como una herramienta más incorporada al proyecto, veamos este ejemplo simple del archivo *mongoose.js*:

```
import mongoose from 'mongoose';

const url = 'mongodb://localhost:27017/'; // String de conexión local
const dbName = 'odm-test'; // Nombre de la base de datos

// Definición del esquema
const exampleSchema = new mongoose.Schema({
  name: {
    type: String,
    required: true,
    indexedDB: true
  },
  age: {
    type: Number,
    required: true
  },
}, {
  timestamps: true // Añade campos createdAt y updatedAt automáticamente
});

// Creación del modelo (equivalente a una colección)
const ExampleModel = mongoose.model('documents', exampleSchema);

async function main() {
  // Conexión a MongoDB usando Mongoose
  await mongoose.connect(`${url}/${dbName}`);
  console.log('Conexión exitosa a MongoDB con Mongoose');

  const insertResult = await ExampleModel.insertMany([ // Esta sintaxis
    depende de Mongoose pero suele ser muy parecida
    { name: 'Ana', age: 28 },
    { name: 'Luis', age: 34 },
    { name: 'María', age: 22 },
  ]);

  console.log('Documentos insertados =>', insertResult);

  return 'Completado.';
}
```

```
}

try {
  const result = await main();
  console.log(result);
} catch (error) {
  console.error('Error en la conexión o ejecución:', error);
} finally {
  // Cierre de conexión
  await mongoose.connection.close();
}
```

Para realizar la prueba:

```
node mongoose.js
```